



COMPETITOR PAYWALL INTELLIGENCE · COUNTRY BY COUNTRY

Reverse-engineer any competitor's paywall — in every market, automatically.

An autonomous agent that drives **real Android devices** through competitor apps to their paywalls — switching exit country per run, reading each screen with vision, and logging a structured teardown to a spreadsheet. It **learns the route** to each paywall as it goes, so repeat runs need no human and almost no model calls.

24

TARGET COUNTRIES

4

APPS TAUGHT

136

REAL PAYWALL
CAPTURES

~12,500

LINES OF PYTHON

WHAT IT DOES

From “what are they charging?” to a filled spreadsheet

Paywall pricing, plan mix, trials and copy differ by **country**, shift with **A/B tests**, and sit **behind onboarding and sign-in**. Doing that teardown by hand — per app, per market — doesn't scale. Monetization Scout automates the whole loop on a real device.

1 · SWITCH

Set the phone's VPN exit country & verify it

2 · REINSTALL

Fresh install so the paywall is re-fetched for that market

3 · DRIVE

Navigate onboarding → sign-in → the paywall

4 · READ

Vision extracts plans, prices, trial, CTA

5 · LOG

One structured row per paywall → Google Sheets

Four core modules

- ◆ **Per-country teardown** on real devices, with honest exit-country verification
- ◆ **Teach-by-recording** — drive an app once by hand; replay it across every market
- ◆ **Vision paywall analysis** → structured rows (plans, prices, trials, tokenomics)
- ◆ **Self-training flow graph** — it learns the route to each paywall over runs

Built & proven

43 modules · 30 releases

SHIPPED ITERATIVELY TO V0.31

4 real apps

TARTEEL/IQRA · AL SIRAAT · MUSLIM PRO · MEMORIZE QURAN

11 / 24

COUNTRIES VERIFIABLE VIA THE CURRENT VPN'S PREMIUM LIST

MODULE 01

Per-country teardown, on a real phone

For each market the agent drops the tunnel for a fast install, reinstalls the app clean so the paywall is fetched **for that country**, switches the VPN to the target exit and **verifies** it, then drives the app to every paywall surface and captures it.

Vision turns each screen into a precise record: plan names, prices, which plan is **highlighted or preselected**, trial framing, CTA text, and how hard the paywall is to dismiss — **one row per surface, per country**.

plans & prices

highlighted / preselected

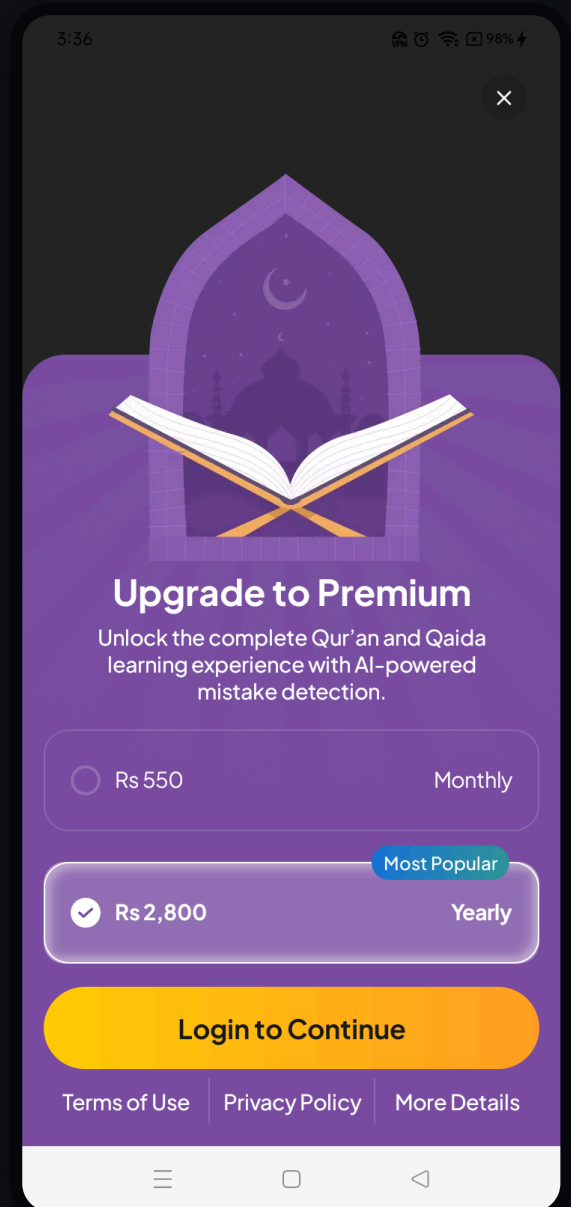
free-trial framing

CTA copy

close-button behaviour

tokenomics / credits

Honesty, by design. Every row is tagged with the verified exit country — a switch that can't be confirmed is logged **UNVERIFIED** rather than trusted, and a screen with no real plans never reaches the sheet.

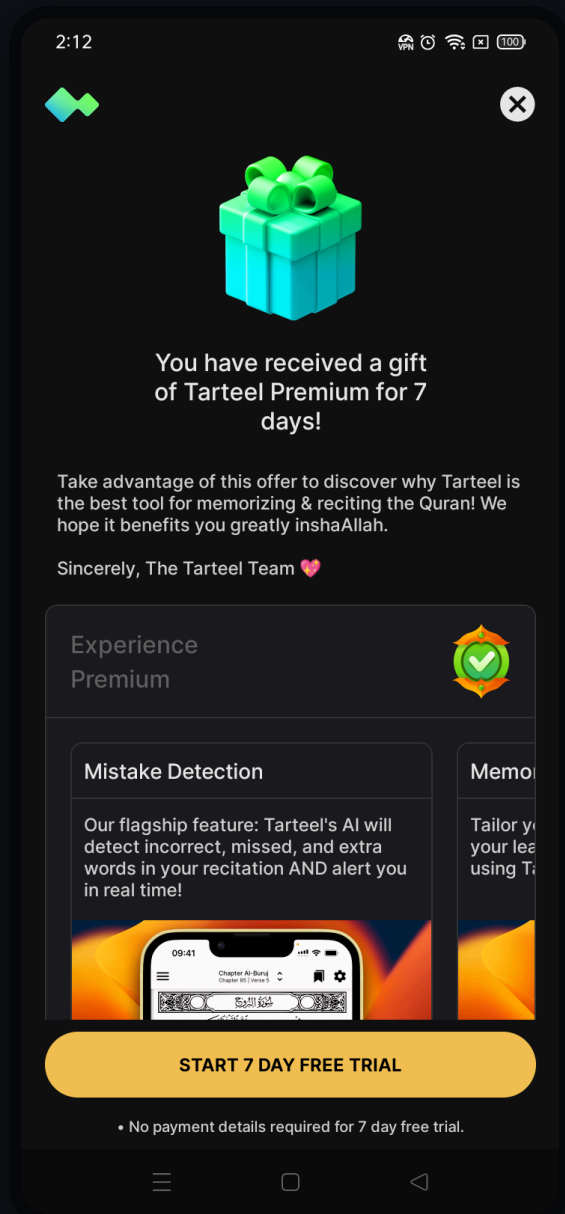


Real capture — Al Siraat “Upgrade to Premium.”

The teardown reads: two plans (Rs 550 monthly, Rs 2,800 yearly), yearly marked *Most Popular* and preselected, CTA “Login to Continue.” Pricing is whatever the app served at capture time.

MODULE 02

Teach it once, replay it everywhere



Flutter apps expose no readable UI tree, so generic automation is blind. The fix is **record mode**: drive the app **once**, by hand, on the phone. Every tap, swipe, back and the pauses between them are captured from the raw touch panel into a reusable recipe — then **replayed across all 24 countries with zero interaction**.

- ◆ Coordinates stored as **screen fractions** — resolution-independent replay
- ◆ Each tap crops the **real button icon** from the screen for an auditable record
- ◆ System dialogs — **Google sign-in**, permission prompts — are part of the flow, fully recorded
- ◆ Taps self-correct a few-pixel layout shift, and wait out real loaders
- ◆ Stuck? A one-click **Icon Picker** lets a human point — costing zero model calls

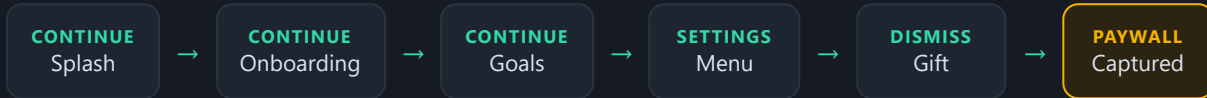
4 apps taught so far, each a 13–29 step recipe that now runs unattended, country after country.

Real capture — an exit-intent “gift” paywall (Tarteel/Iqra). A different paywall shape entirely — a 7-day trial offer — reached and captured by the same recorded recipe.

MODULE 03

It teaches itself the route to the payoff

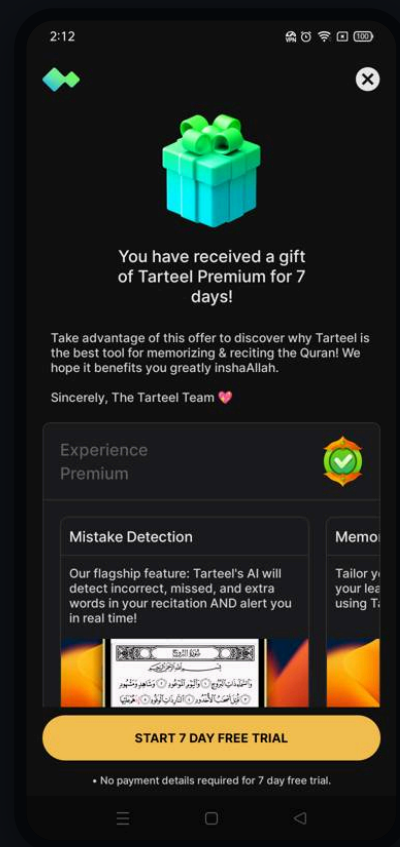
Every tap the agent ever makes — recorded, picked, or explored — becomes training data. From it the tool builds a per-app **flow graph**: **screen** → **the tap that advances** → **next screen**, with a probability that the tap makes progress.



Faithful view of the real learned graph for one app — **11 screens, 13 edges**. Roles (Continue / Settings / Dismiss / Premium) are assigned automatically from labels & position.

- ◆ Once an edge clears **5 hits and 80% progress**, the agent taps it **itself** — no human, no vision call for that step
- ◆ A tap that stops working **decays** back below the bar, so the graph self-heals when an app is redesigned
- ◆ Graphs **pool across machines** — ten people running once is the same as one machine running ten times

The cost angle. The learned graph and the recorded recipes are **free** to run. The paid vision model is the **last** resort — used only on screens nothing cheaper can handle — so cost **falls** the more each app is run.



The screen the graph flags as the payoff — the destination every learned route leads to.

UNDER THE HOOD

The hard problems it had to solve

Locked-down real devices

ColorOS/Realme block the normal reset path and force installer attribution, so the agent reinstalls genuinely through the Play Store and steps past OEM security dialogs to reach a true first-run state.

Flutter's invisible UI

Many targets render to an opaque canvas with no readable elements. Only pixels and coordinates work — which is exactly why record-and-replay and vision, not selectors, are the backbone.

Honest geo, honest data

A phone VPN silently drops packets and can land on the wrong country. The agent verifies the exit market device-side and refuses to trust — or log — an unverified switch.

Crash-safe control

The device gesture engine can crash mid-run, so every tap and swipe goes through **adb** instead; loader-aware gates wait out progress screens rather than racing ahead into a wrong capture.

STACK

Python

Appium · UiAutomator2

adb

Vision model (Groq, OpenAI-compatible)

Google Sheets API

on-device VPN control

PyInstaller desktop build

Honest limitations. Country coverage is bounded by the VPN's server list (11 verifiable today); Google Play prices follow the store account's country, which the write-up calls out per row; and the vision model is the one paid dependency — deliberately kept to the last resort. Screenshots here are real captures from test runs, not sample data.